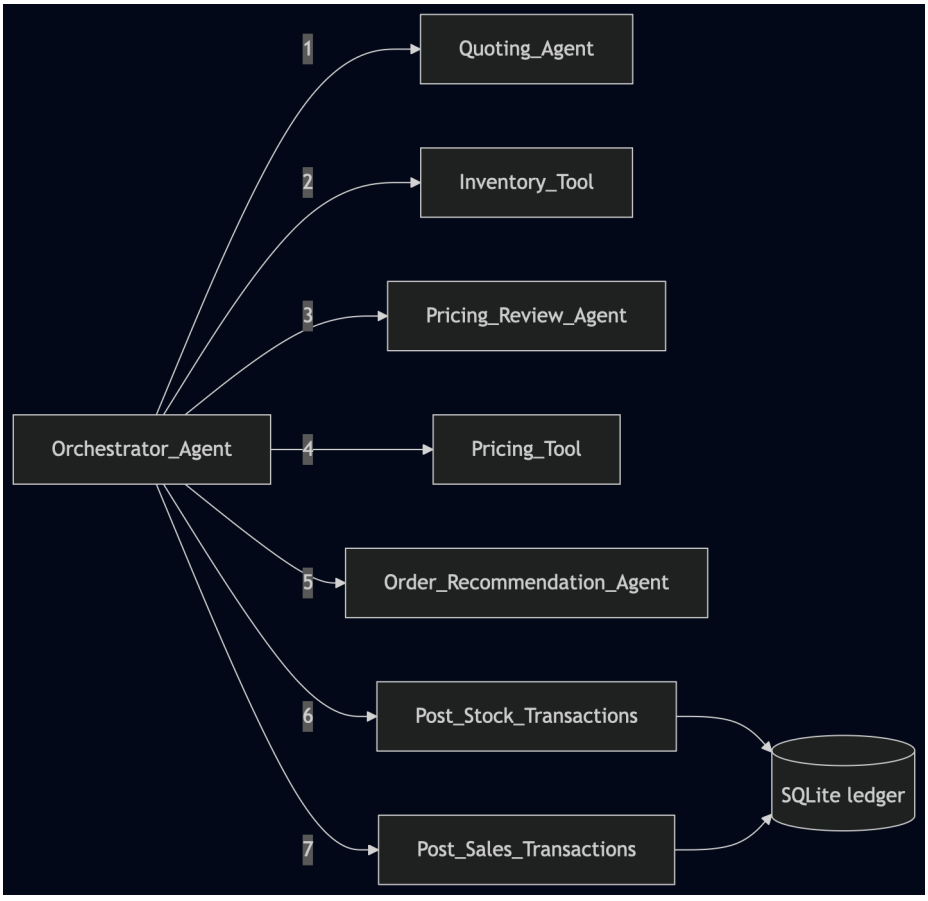
Agent Workflow Architecture and Orchestration

Architecture & Responsibility Separation

The multi-agent system was architected around a centralized Orchestrator Agent that managed task delegation using an agentic ReAct (Reasoning and Acting) loop. Specifically the **Orchestrator Agent** was responsible for taking the natural language text quote and running appropriate tools and agents to process the quote into a purchase and provide a response back to the customer. See diagram below.



Note: The Quoting Agent,

1. Quoting_Agent

- **Operational Model: Agentic**
- **Data Input:** Unstructured customer inquiry string
- **Data Output:** A structured QuoteResponse object containing date_of_request, need_date, and product name and quantity requested.
- **Justification for Agentic Model:** Agents are really good at processing human text inputs that possess infinite semantic variations (e.g., *"Gimme 5 packs of a4"* vs *"Requesting fifty boxes of legal paper for next Tuesday"*).

2. Inventory_Tool

- **Operational Model: Deterministic**
- **Data Input:** Structured array of line items from QuoteResponse, target arrival dates.
- **Data Output:** An InventoryResult structure detailing exact stock quantities on hand, calculated shortfalls, required procurement reorder units and delivery dates from suppliers. In addition a boolean flag is provided to clarify if the order can be processed within the required delivery date.
- **Starter Code:** get_stock_level() and get_all_inventory().
- **Justification for Deterministic Model:** Checking quantities is an exact mathematical problem. Using an agent enabled introduced a risk of hallucination where accuracy is mandatory.

3. Pricing_Review_Agent

- **Operational Model: Agentic**

- **Data Input:** Validated product identifiers extracted from the active quoting sequence.
- **Data Output:** A PricingReviewResponse object containing statistical historical price bands (minimum, average, and maximum unit prices calculated from past business records).
- **Starter Code Helper Mapping:** search_quote_history().
- **Justification for Model Hybridization:** While the database query itself is deterministic, historical entries often contain human logging errors, acronyms, or product spelling variations. An agentic boundary safely resolves semantic differences across historic log texts while returning true numbers.

4. Pricing_Tool

- **Operational Model: Deterministic**
- **Data Input:** Validated historical price bands, calculated volume acquisition requirements, and current corporate capital reserves.
- **Data Output:** The pricing tool output discrete math calculations for the following three strategies:
  1. **maximize_profit**: Calculated using historical maximum unit price bounds (max_unit_price multipliers).
  2. **average_pricing**: Calculated using historical average unit price fields (average_unit_price).
  3. **maximize_turnover**: Calculated using historical minimum unit price bounds (min_unit_price) to stimulate quick conversion.

  Critically the tool provided a boolean to clarify if there was enough capital to cover ordering the products. Upon fail this

allowed the orchestrator agent to prevent procuring the stock and ending up with a negative cash balance.

- **Starter Code Helper Mapping:** get_cash_balance()
- **Justification for Deterministic Model:** These were deterministic calculations. Enabling these calculations for a agent would have enabled hallucinations.

5. Order_Recommendation_Agent

- **Operational Model: Agentic**
- **Orchestrator Data Input:** Compilation of the upstream QuoteResponse, InventoryResult, evaluated pricing matrices, and raw context parameters.
- **Orchestrator Data Output:** Selected target pricing profile strategy, final line-item execution approvals, and a tailored customer response.
- **Starter Code Helper Mapping:** generate_financial_report().
- **Justification for Agentic Model:** Selecting a final strategy and authoring client communications requires situational judgment. The agent balances corporate context against user needs to decide whether to prioritize relationship-building (turnover strategy) or revenue (profit strategy), generating professional prose justifying the corporate outcome.

6 Post_Stock_Transactions

- **Operational Model: Deterministic**
- **Orchestrator Data Input:** A collection of approved replenishment records from the Order Recommendation output where stock shortfalls require third-party sourcing.

Input parameters include item_name, quantity (representing qty_to_order), wholesale price (representing unit_cost), and the execution transaction date set to the historical date_of_request.

- **Data Output:** Not applicable. This function executed functions within the starter_project.py code to finalize a transaction.
- **Starter Code Helper Mapping:** Wraps and executes create_transaction().
- **Justification for Deterministic Model:** This tool updated the database to reflect procurement transactions. Allowing an agentic component to post stock orders dynamically runs the risk of hallucinations for functions that can be purely deterministic.

7. Post_Sales_Transactions

- **Operational Model: Deterministic**
- **Orchestrator Data Input:** A collection of approved customer line items containing item_name, calculated retail quantity (representing qty_fulfilled), final strategy-derived retail price (representing customer unit_price), and the execution transaction date set precisely to the customer's requested need_date.
- **Orchestrator Data Output:** Not applicable. This function executed functions within the starter_project.py code to finalize a transaction.
- **Starter Code Helper Mapping:** Wraps and executes create_transaction().

- **Justification for Deterministic Model:** This tool updated the database to reflect financial sales transactions. Allowing an agentic component to post sales transactions dynamically runs the risk of hallucinations for functions that can be purely deterministic.

3. Reflection and Architecture Analysis

3.1 Architectural Choices and Decision-Making

The system relies on an Orchestrator-Worker pattern designed around structural execution gates. During development many of the functions were tried with agentic calls rather than deterministic code. Clear trade offs were identified between what should be "deterministic" and what should be "agentic" based on strengths for the two approaches. For a transaction system there are many functions that are deterministic where an adding an agent just adds overhead.

3.2 Strengths of the Implemented System

System evaluation logs highlight two primary operational strengths:

1. **State Protection via Early Exit Gates:** The system demonstrates high resilience by blocking execution pipelines the moment a tooling error or asset shortfall is encountered, protecting database integrity from invalid transactional entries.
2. **Deterministic Fallback Selection:** When capital constraints limit purchasing power, the Orchestrator switches dynamically to algorithmic optimization—prioritizing line items that yield the highest profitability per dollar spent to maintain business performance.

3.3 Suggestions for Further Improvements

I would recommend investigating other LLM Models to find best fit. This agentic workflow was built with gpt-5.4-mini. If more sophisticated models were used it could be possible to simplify the agentic workflow with fewer, more sophisticated agents. This could be compared to the average cost of executing a quote to find the best fit solution that maximized simplicity while minimizing cost.